TRƯỜNG ĐẠI HỌC VĂN LANG · KHOA CÔNG NGHỆ THÔNG TIN

BUỔI 10 · THỰC HÀNH

Workshop xây dựng module mở rộng và Giao diện — Sprint 2

Tích hợp các module thành luồng chạy end-to-end cục bộ

Giảng viên chính HK261: **TS. Nguyễn Trí Hải / ThS. Nguyễn Minh Tân / ThS. Trần Công Thanh**

Học kỳ 1, năm học 2026 – 2027

MỤC TIÊU BUỔI HỌC

Luồng chính phải chạy được end-to-end

1

Module mở rộng theo track đã hiện thực

[SE] Frontend ≥ 3 màn hình kết nối API thật · [DA] Nạp kho dữ liệu và dashboard ≥ 2 biểu đồ · [AI] Baseline + ≥ 2 mô hình đã huấn luyện, có metric.

2

Các module đã tích hợp: luồng chính chạy từ đầu đến cuối

Chạy một lần từ điểm bắt đầu tới điểm kết thúc mà không phải can thiệp tay giữa chừng và không lỗi treo. Đây là mục tiêu quan trọng nhất hôm nay.

3

≥ 5 unit test chạy PASS

BT2 yêu cầu ≥ 3 test thuộc ba loại (luồng đúng · giá trị biên · dữ liệu sai); đặt mục tiêu 5 hôm nay để có dư. Test cho lớp nghiệp vụ, không phải test giao diện.

4

README đã cập nhật đủ để người khác chạy lại

Mục 3 (các bước dựng hệ thống, mỗi bước một lệnh) phải điền xong hôm nay và phải được thử trên một thư mục clone mới.

NHỊP BUỔI HỌC

Bốn chặng

1

0 – 10 phút
Mở đầu

GV nhắc checklist và cảnh báo tiến độ. SV nêu một câu về chỗ mình đang kẹt.

2

10 – 70 phút
Module mở rộng

Hiện thực phần mở rộng theo track. GV đi hết lớp, ưu tiên nhóm chưa chạy được ở buổi 8.

3

70 – 110 phút
Tích hợp

Ghép các module thành luồng chạy end-to-end. Chạy thử toàn luồng, sửa chỗ đứt gãy.

4

110 – 135 phút
Test và README

Viết unit test cho tới khi đủ 5 test pass; cập nhật README; peer review: bạn bên cạnh clone và chạy theo README.

Chặng 4 hay bị cắt vì hết giờ — đừng cắt. Một repo chạy được nhưng README sai thì vẫn trượt mục kiểm chứng cốt lõi của BT2. Nếu thiếu thời gian, hãy cắt bớt phạm vi ở chặng 2 chứ không cắt chặng 4.

CHẶNG 2 · MODULE MỞ RỘNG THEO TRACK

Nhiệm vụ

SE Giao diện gọi API thật

- Tạo src/api/client.js và một tệp *.api.js cho nghiệp vụ phiếu
- Dựng 3 màn hình theo wireframe BT1: danh sách, chi tiết, tiếp nhận
- Bọc mỗi màn hình bằng thành phần DataState (đang tải · rỗng · lỗi)
- Đăng nhập; nhận 401 thì xoá token, về trang đăng nhập
- **Xong khi:** tắt backend → thấy thông báo lỗi có nút thử lại; tìm “fetch(” trong pages/ ra rỗng

DA Kho dữ liệu và dashboard

- Chạy nạp dimension bằng khoá nghiệp vụ, rồi nạp fact theo lô ngày
- Đếm dòng staging và fact; giải thích chênh lệch
- Dựng 2 biểu đồ, mỗi biểu đồ gắn một câu hỏi phân tích
- Thêm mốc tham chiếu và dòng ghi nguồn, thời điểm dữ liệu
- **Xong khi:** nạp lần hai không nhân đôi dòng; bộ lọc rỗng hiện câu giải thích

AI So sánh và đánh giá mô hình

- Huấn luyện DummyClassifier và ít nhất 2 mô hình thật với cùng hạt giống
- Chọn mô hình trên validation; đo trên test đúng MỘT lần
- Ghi metrics.json: Macro F1, F1 từng nhãn, confusion matrix
- Viết ba câu kết luận đọc từ confusion matrix
- **Xong khi:** chạy hai lần cho metrics.json giống nhau; .gitignore chặn tệp mô hình

KHI THIẾU THỜI GIAN

Thu hẹp phạm vi đúng cách — cắt gì, giữ gì

ĐƯỢC cắt hoặc hoãn

- Màn hình thứ tư trở đi, chức năng phụ ngoài luồng chính
- Giao diện đẹp: màu sắc, hiệu ứng, biểu tượng
- Biểu đồ thứ ba trở đi [DA]; mô hình thứ ba trở đi [AI]
- Tối ưu hiệu năng khi dữ liệu còn nhỏ
- Tính năng “hay” chưa có trong User Story của BT1

KHÔNG được cắt

- Luồng chính chạy end-to-end trên dữ liệu thật
- Ba trạng thái cho ít nhất một màn hình hoặc báo cáo
- ≥ 3 unit test thuộc ba loại: luồng đúng · giá trị biên · dữ liệu sai
- README mục 3 đã thử trên thư mục clone mới
- Ghi nhận phần đã cắt và lý do trong mục điều chỉnh thiết kế

Cắt phạm vi có ghi nhận là quyết định kỹ thuật có trách nhiệm (CLO5). Cắt README hoặc test để kịp thêm tính năng là lỗi ưu tiên — bảng tiêu chí BT2 chấm README và test, không chấm số lượng tính năng.

Sáu bước để luồng chính chạy end-to-end

- [ ] BƯỚC 1 — Viết ra luồng chính bằng lời, đánh số các bước. Dùng lại luồng chính trong đặc tả use case đã nộp ở BT1 — không cần nghĩ mới.
- [ ] BƯỚC 2 — Chạy thử từng bước RIÊNG LẺ trước. Bước nào chưa chạy được thì sửa trước khi ghép. Đừng ghép hết rồi mới chạy.
- [ ] BƯỚC 3 — Ghép hai bước liền kề, chạy thử. Rồi ghép thêm bước thứ ba, chạy thử. Ghép tăng dần, không ghép một lần.
- [ ] BƯỚC 4 — Với mỗi chỗ đứt gãy, xác định đó là lỗi HỢP ĐỒNG DỮ LIỆU (hai bên hiểu khác nhau về hình dạng dữ liệu) hay lỗi LOGIC. Hai loại này sửa khác nhau.
- [ ] BƯỚC 5 — Chạy toàn luồng từ đầu tới cuối trên dữ liệu mẫu THẬT, không phải dữ liệu giả. Ghi lại đầu vào và đầu ra quan sát được.
- [ ] BƯỚC 6 — Chụp ảnh màn hình kết quả toàn luồng, lưu vào docs/. Đây là minh chứng cho mục kiểm chứng BT2.

Bước 3 (ghép tăng dần) là bước tiết kiệm thời gian nhất. Ghép hết rồi mới chạy sẽ cho ra một lỗi mà không biết nó đến từ đâu trong sáu bước.

Gặp triệu chứng này thì kiểm tra gì trước

Track	Triệu chứng	Nguyên nhân thường gặp	Kiểm tra trước tiên
SE	Trình duyệt báo lỗi CORS hoặc gọi API ra 404	Sai base URL, sai cổng, backend chưa bật CORS cho cổng của frontend	Gọi thử API bằng curl hoặc Postman; xem .env và cấu hình CORS
SE	Màn hình hiện “undefined” ở một cột	Lệch hợp đồng: backend trả ten_khach, frontend đọc tenKhach	In đối tượng tại hàm chuyển đổi; so với comment hợp đồng
DA	Số dòng fact ít hơn staging	JOIN loại dòng có khoá không khớp dimension	Truy vấn dòng staging không tìm thấy khoá; thêm dòng “không xác định”
DA	Dashboard hiện số cũ sau khi nạp lại	Cache của Streamlit hoặc Power BI chưa làm mới	Xoá cache hoặc giảm ttl; ghi thời điểm dữ liệu lên biểu đồ
AI	Metric tốt bất thường (F1 > 0,95)	Rò rỉ dữ liệu: trùng mẫu giữa train và test, hoặc đặc trưng chứa nhãn	Kiểm tra trùng mô tả giữa các tập; xem đặc trưng quan trọng nhất
AI	API suy luận cho kết quả khác lúc đánh giá	Tiền xử lý lúc suy luận khác lúc huấn luyện	Dùng chung build_features(); lưu vectorizer cùng mô hình

Quy tắc chung: lỗi “undefined / None” thường là lỗi hợp đồng; lỗi “số ra không đúng” thường là lỗi logic hoặc ý nghĩa. Buổi 11 sẽ học quy trình debug đầy đủ.

Viết test cho quy tắc nghiệp vụ, không viết test cho giao diện

Ví dụ: test cho quy tắc QT-05 (còn bảo hành hay không)

```
// tests/ticket.service.test.js
describe('kiemTraConBaoHanh — QT-05', () => {

  test('còn bảo hành khi mua chưa quá số tháng bảo hành', () => {
    const tb = { ngayMua: '2026-01-15', soThangBaoHanh: 12 };
    const r = kiemTraConBaoHanh(tb, '2026-09-08');
    expect(r.conBaoHanh).toBe(true);
  });

  test('hết bảo hành khi đã quá số tháng bảo hành', () => {
    const tb = { ngayMua: '2024-01-15', soThangBaoHanh: 12 };
    const r = kiemTraConBaoHanh(tb, '2026-09-08');
    expect(r.conBaoHanh).toBe(false);
  });

  // TEST QUAN TRỌNG NHẤT: trường hợp biên / ngoại lệ đã đặc tả ở BT1
  test('chưa xác minh được khi không có ngày mua', () => {
    const tb = { ngayMua: null, soThangBaoHanh: 12 };
    const r = kiemTraConBaoHanh(tb, '2026-09-08');
    expect(r.conBaoHanh).toBeNull();
    expect(r.lyDo).toBe('CHUA_XAC_MINH_BAO_HANH');
  });
});
```

Ghi nhớ

- Ba test cho một quy tắc: trường hợp đúng, trường hợp sai, và trường hợp BIÊN.
- Trường hợp biên là test có giá trị nhất — nó chính là luồng ngoại lệ Sinh viên đã đặc tả ở BT1.
- Test cho lớp NGHIỆP VỤ, không cần cơ sở dữ liệu thật — đó là lý do tách lớp repository.
- Tên test viết bằng lời, mô tả hành vi mong đợi. Người đọc hiểu quy tắc mà không cần đọc mã.
- [DA] Tương đương: test hàm chuẩn hoá ngày với ba định dạng + chuỗi rác.
- [AI] Tương đương: test hàm build_features trả đúng số cột và đúng thứ tự cột.

CHẶNG 4 · PEER REVIEW NGẮN · 10 PHÚT

Trao đổi sinh viên khác chạy repo theo README

1

Đổi vai (1 phút)

Hai Sinh viên ngồi cạnh nhau gửi cho nhau đường dẫn repo. Mỗi người chỉ được đọc README, không được hỏi chủ repo.

2

Clone và chạy theo README (6 phút)

Clone vào thư mục mới, làm đúng từng bước trong README mục 3. Ghi lại bước nào thiếu, lệnh nào lỗi, chỗ nào phải đoán.

3

Kiểm ba thứ (2 phút)

Luồng chính có chạy end-to-end không? Lệnh chạy test có hiện số test pass không? Có màn hình hoặc báo cáo nào xử lý trạng thái rỗng và lỗi không?

4

Ghi phản hồi (1 phút)

Viết ba dòng vào issue của repo: một điều chạy tốt, một bước README cần sửa, một rủi ro cho BT2. Chủ repo sửa README ngay trong buổi.

Peer review không chấm điểm cho nhau — mục đích là phát hiện bước README bị thiếu khi vẫn còn hai buổi để sửa.

CHECKLIST BUỔI 10 – ĐỐI CHIẾU TRƯỚC KHI KẾT THÚC BUỔI

Mười mục kiểm chứng của buổi hôm nay

- [ ] Module mở rộng theo track đã hiện thực và chạy được.
- [ ] Luồng chính chạy END-TO-END không lỗi treo.
- [ ] Đã chạy trên dữ liệu mẫu THẬT, không phải dữ liệu tự gõ vài dòng.
- [ ] Đã chụp ảnh kết quả toàn luồng, lưu vào docs/.
- [ ] ≥ 5 unit test chạy PASS; lệnh chạy test hiển thị số test trên màn hình.
- [ ] Có ít nhất 1 test cho trường hợp BIÊN hoặc luồng ngoại lệ.
- [ ] Ba trạng thái (đang tải, rỗng, lỗi) đã xử lý cho ít nhất một màn hình hoặc báo cáo.
- [ ] README mục 3 đã điền đủ các bước chạy, mỗi bước một lệnh.
- [ ] ĐÃ THỬ clone repo vào thư mục mới và chạy theo đúng README.
- [ ] Đã commit theo quy ước; tổng số commit của repo đã ≥ 8.

Ba mục tô đỏ là ba mục quyết định. Mục 9 là mục sinh viên hay bỏ nhất và cũng là mục khiến trượt mục kiểm chứng cốt lõi của BT2 — hãy làm ngay hôm nay, đừng để tới buổi 12.

KẾT THÚC BUỔI HỌC

Chốt buổi 10 và chuẩn bị cho buổi 11

Tự học, tự nghiên cứu

- Hoàn thành các mục còn thiếu trong checklist 10 mục.
- THỬ clone repo vào thư mục mới và chạy theo README — nếu chưa làm ở lớp thì làm ngay hôm nay.
- Bổ sung unit test cho tới khi đủ 5 test pass.
- Đọc tài liệu tự học buổi 11 theo track: đóng gói và cấu hình theo stack.
- Cài Docker Desktop

Buổi 11 (Lý thuyết) — Tích hợp hệ thống & Xử lý lỗi

- Nguyên tắc tích hợp; quản lý phụ thuộc và khoá phiên bản.
- Debug có hệ thống: tái lập → khoanh vùng → sửa → xác nhận.
- Logging: ghi gì, ghi ở mức nào, và tuyệt đối không ghi gì.
- Docker cơ bản: Dockerfile và docker-compose.
- Đây là buổi lý thuyết CUỐI trước hạn nộp Bài Tập 2.